JobFit

Ranking entry-level job postings by transparent, repeatable requirement coverage

Final Project Report — Generative AI

Group members: [add names and UNIs]

Code: [add GitHub repository URL]

November 2026

Abstract

Entry-level candidates typically mass-apply with one generic resume and get no feedback on why applications fail. Pasting a resume and a job description into a chatbot helps for one posting at a time, but its scores vary between runs, it does not show what a score is based on, and it cannot rank thousands of postings. **JobFit** is an API with a web demo that answers one question: *given my resume, which jobs should I apply to?* It (1) converts every job description and the resume into a fixed structured schema (LLM extraction with a JSON schema, or a deterministic rule-based fallback), (2) links each stated requirement to the closest resume evidence with exact taxonomy matching plus calibrated sentence-embedding similarity, (3) combines requirement coverage, experience, education and role alignment with an explicit formula into a 0–100 score with sub-scores, and (4) explains the top results using only the structured match output. The pipeline is served by FastAPI in a Docker container and scores a resume against all 8,000 LinkedIn postings in our database in 146 ms (median) on a CPU.

On 586 Kaggle resumes with weak relevance labels, JobFit reaches nDCG@10 = 0.701, above keyword TF-IDF (0.695) and exact skill overlap (0.673) but below whole-document embedding similarity (0.742); a hybrid of the two reaches 0.739. Semantic matching raises recall on paraphrased evidence from 38% (keywords) to 69% with no false positives on hard negatives; scores are identical across repeated runs and move in the expected direction in 100% of counterfactual edits. We discuss what the weak labels can and cannot measure, and the limitations of the system.

1. Introduction

Graduate students in campus recruiting see the same pattern: the entry-level market is competitive, so most students send one generic resume to as many postings as possible. This takes hours each week, produces few responses and gives no feedback. Generic resumes are also more likely to be filtered by applicant-tracking systems. A better strategy is **targeted applying**: concentrating effort on roles where the candidate is a realistic fit.

The common shortcut — pasting a resume and a job description (JD) into ChatGPT — has three structural problems: it handles **one posting at a time**, its **scores change between runs**, and it **does not show what the score is based on**. JobFit is designed around these gaps. Its score is defined as *how well the resume covers the posting's stated requirements*; it is explicitly **not** a probability of getting an interview.

Contributions.

- A structured-extraction layer with a fixed schema (requirements with category and required/preferred importance, minimum years, minimum degree and fields), implemented twice: with an LLM constrained by a JSON schema at temperature 0 and cached, and with a deterministic 115-skill taxonomy plus section heuristics that runs offline.
- A requirement-to-evidence matcher that combines exact taxonomy matches with calibrated sentence-embedding similarity, upgrading the word-vector approach from the course (Module 2) to sentence-level embeddings; descriptor-augmented requirement embeddings raise positive/negative separability (AUC) from 0.920 to 0.947.

- A transparent, repeatable scoring formula, implemented both per pair (with evidence) and vectorised over the whole job database for ranking at scale, with a test that guarantees both agree.
- A FastAPI service (`/match`, `/rank`, `/rank/upload`, …) extending the Module 3 service, a web demo, and a Docker image that runs fully offline.
- An evaluation with baselines, ablations, a paraphrase benchmark, counterfactual behaviour tests and stability/latency measurements, plus a script for a measured comparison with the plain-LLM approach.

2. Data

2.1 Job postings

The job database comes from the Kaggle **LinkedIn Job Postings (2023–2024)** dataset (123,849 postings, with a companion table of LinkedIn job-function tags). We kept postings whose experience level is *Entry level*, *Associate* or *Internship* (47,983), then postings tagged with at least one target function — Finance (FIN), Accounting/Auditing (ACCT), Information Technology (IT), Consulting (CNSL) or Analyst (ANLS) — and with a description longer than 300 characters (12,065). Because IT accounts for about two thirds of these, we kept every posting with a non-IT target tag and filled the remainder with randomly sampled IT-only postings (seed 42), giving **8,000 postings**: 5,079 entry level, 2,596 associate and 325 internships. Function counts (multi-label): IT 4,147, FIN 2,559, ACCT 1,620, ANLS 1,179, CNSL 573.

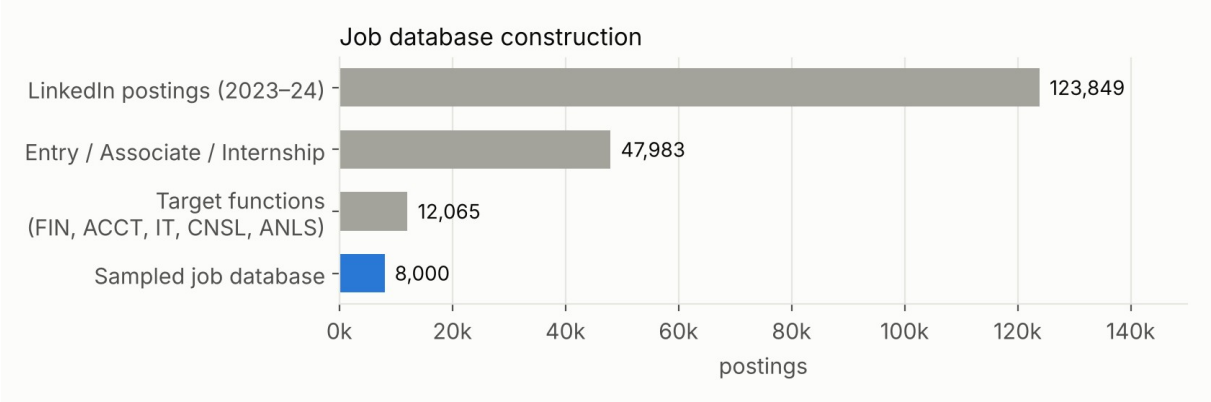


Figure 1. Construction of the 8,000-posting job database.

Descriptions were lightly normalised (whitespace, and re-inserting the sentence breaks LinkedIn often drops, e.g. “transactions.Oversee” → “transactions. Oversee”); an optional `skills_desc` field is appended. We store title, company, location, level, work type, remote flag, salary and URL for filtering and display.

2.2 Resumes

The **Kaggle Resume Dataset** contains 2,484 resumes in 24 categories. We use the 586 resumes in the five categories that correspond to our job functions — ACCOUNTANT (118), FINANCE (118), BANKING (115), INFORMATION-TECHNOLOGY (120) and CONSULTANT (115) — for testing and evaluation only; nothing is trained on them. The raw text encodes bullet and section boundaries as

runs of spaces, which we convert to line breaks so that each bullet becomes an evidence unit. Two properties matter for interpreting results: most resumes describe **experienced** professionals rather than students (median parsed experience is well above entry level), and categories are coarse (many CONSULTANT resumes are IT consultants).

2.3 Data preparation is reproducible

`scripts/download_data.sh` → `scripts/prepare_data.py` → `scripts/build_index.py` regenerate everything from the public datasets. The processed database and the pre-built index (~19 MB) are committed so the API and Docker image work without Kaggle access.

3. System design

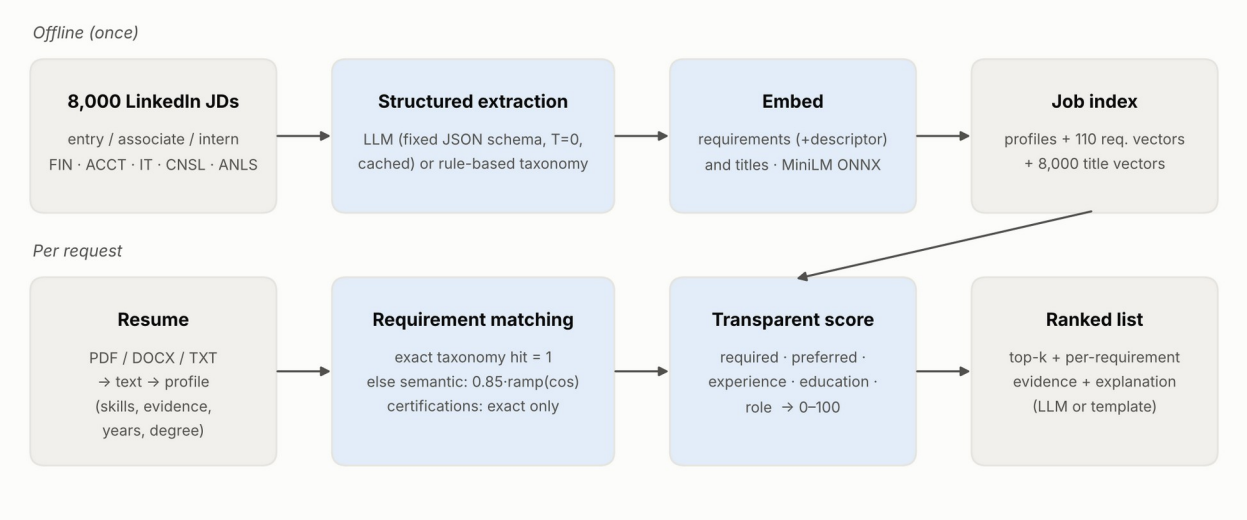


Figure 2. JobFit architecture. JD extraction and embedding run once; per request only the resume is processed and every posting is scored.

3.1 Structured extraction

Both the JD and the resume are mapped onto fixed pydantic schemas (Table 1). The schema is the contract between components: matching, scoring, explanations and the API depend only on it, so the extractor can be swapped without touching the rest.

Object	Fields
Requirement	text (≤12 words) · category (skill / tool / certification / domain / other) · importance (required / preferred) · canonical skill
JD profile	requirements[] · min_years_experience · education_level (none … doctorate) · education_fields[]
Resume profile	skills[] (canonical) · evidence[] (bullets / sentences) · years_experience · education_level · education_fields[]

Table 1. Extraction schema (app/schemas.py).

LLM extractor. The LLM (Gemini, called through its OpenAI-compatible endpoint so any OpenAI-compatible model can be substituted) receives the posting and an instruction to list only qualifications — not duties or benefits — with importance *required* unless the posting marks it

preferred, and to put years and degrees in their own fields. Output is constrained by `response_format: json_schema`, temperature is 0, and every (model, prompt) pair is cached in SQLite, so each JD is extracted exactly once. Free-text skill names are canonicalised onto the taxonomy so exact matching still applies. Any failure falls back to the rule-based extractor.

Rule-based extractor. A 115-skill taxonomy over finance, accounting, banking, IT, data, consulting and soft skills (each with aliases, e.g. *SQL* ← postgresql, t-sql) is matched on word boundaries. Each JD segment is labelled by section — requirement, preferred, duty, benefit, other — from heading cues ("Requirements", "Preferred qualifications", "What you'll do", "Benefits") and inline cues ("… is a plus"). Skills in a requirement list are required; skills only in duties are preferred when an explicit requirement list exists; if that would leave no required skill, duty skills are promoted. Minimum years come from patterns such as "3+ years of … experience"; resume years are the union of employment date ranges with overlaps merged. Degrees and fields are parsed with regular expressions. On the 8,000 postings the extractor finds on average about 9 requirements; 129 postings (1.6%) yield none and receive the prior coverage.

3.2 Semantic matching

For each requirement we look for the strongest resume evidence. Resume evidence units are bullets/sentences (at least 3 words) plus the canonical skills found in the resume. An **exact** taxonomy hit gives strength 1. Otherwise we take the maximum cosine similarity between the requirement and any evidence unit, using `all-MiniLM-L6-v2` sentence embeddings run on ONNX Runtime (no GPU or PyTorch needed), and map it through a linear ramp:

$$\text{semantic}(r) = s_{max} \cdot \text{clip}((\max_e \cos(r, e) - lo) / (hi - lo), 0, 1), \quad s_{max} = 0.85$$

Related evidence is therefore worth at most 85% of direct evidence, and certifications/licences (CPA, CFA, Series 7) are exact-only — a licence is either held or not. Two design details matter:

- **Descriptor-augmented requirements.** A bare tool name carries little meaning for a sentence encoder: "Power BI" vs "Built interactive Tableau dashboards" has cosine 0.15. Embedding "Power BI (business intelligence dashboards and data visualization)" raises it to 0.55. Every skill is embedded with a short descriptor (hand-written for 33 tools, aliases otherwise).
- **Data-driven calibration.** lo and hi are set from 2,629 positive pairs (a skill and a resume sentence that mentions it) and 3,000 negative pairs (a skill and a sentence with no skill from the same group): lo = 95th percentile of negatives (0.324), hi = max(75th percentile of positives, lo + 0.2) (0.524).

Embedding model	Bare skill name AUC	With descriptor AUC	Calibrated lo / hi
all-MiniLM-L6-v2 (selected)	0.920	0.947	0.324 / 0.524
bge-small-en-v1.5	0.855	0.900	0.631 / 0.831

Table 2. Calibration: probability that a positive pair scores higher than a negative pair (AUC).

3.3 Scoring formula

All constants are in `app/config.py`; nothing is learned, so every number in the output can be traced back to the formula.

$$w_i = \text{importance_weight}(r_i) \times (0.5 \text{ if } r_i \text{ is a soft skill else } 1), \quad \text{importance_weight: required } 1, \text{ preferred } 0.5$$

$$\text{coverage}(S) = \left(\sum_{i \in S} w_i \cdot \text{strength}_i + k \cdot p\right) \div \left(\sum_{i \in S} w_i + k\right), \quad k = 2,\ p = 0.3$$

$$\text{experience} = \min\left(1, \frac{\text{years}_{\text{resume}}}{\text{years}_{\text{JD}}}\right); \quad \text{education} = 0.6 \cdot \text{level_fit} + 0.4 \cdot \text{field_fit}$$

$$\text{role} = \text{ramp}\left(0.5 \cdot \cos(\text{title}, \text{resume headline}) + 0.5 \cdot \text{mean top-3} \cos(\text{title}, \text{evidence})\right)$$

$$\text{overall} = 100 \cdot \left(\sum_c W_c \cdot \text{sub}_c\right) \div \left(\sum_c W_c\right), \quad \text{summing only over sub-scores that are defined}$$

Smoothing (*k* pseudo-requirements at prior *p*) prevents a posting with a single matched requirement from scoring 100. Soft skills are down-weighted because nearly every resume claims them. Sub-scores that do not apply (a JD with no stated years) are dropped and the weights renormalised, rather than counted as zero. Over-qualification is not penalised. Education level fit is 1 if the resume meets the minimum degree, 0.5 if one level below; field fit gives partial credit (0.7) within a family (e.g. accounting ↔ finance).

Component	Weight W_c	What it measures
Required coverage	0.45	Weighted, smoothed strength of evidence for required qualifications
Preferred coverage	0.10	Same, for nice-to-have qualifications
Experience	0.15	Resume years vs. the posting's minimum years
Education	0.10	Degree level and field vs. the posting's minimum
Role alignment	0.20	Similarity between the job title and the resume's headline/evidence

Table 3. Sub-scores and weights.

Each requirement is reported with status **matched** (strength ≥ 0.75), **partial** (≥ 0.35) or **missing**, the method (exact / semantic) and the resume sentence used as evidence (Figure 3).

3.4 Ranking at scale

The per-pair scorer is clear but slow in Python. For ranking, all requirements of all 8,000 postings are laid out in compressed-sparse-row arrays; the requirement texts are deduplicated (110 unique texts with the rule extractor), so one matrix product between the unique requirement embeddings and the resume evidence embeddings gives every requirement's best similarity, and per-posting coverage is computed with `bincount`. Ranking all postings takes 146 ms median (p95 246 ms) including resume parsing and embedding; only the top-k are then re-scored in detail for evidence and explanations. Re-posts with the same title and company are de-duplicated in the results. A test asserts that the vectorised and per-pair scores agree for random postings.

3.5 Explanations

The top results receive a short explanation. The primary design uses an LLM whose prompt contains **only the structured match output** (requirements, status, evidence snippet, sub-scores) — not the raw resume or JD — so it can only talk about requirements that were actually matched or missed. The deterministic fallback composes the same information with a template: band and score, how many required qualifications are covered (naming the strongest), related-but-not-exact evidence with a quoted snippet, missing requirements, and the experience comparison. The template is free, consistent and is what the Docker image uses by default when no API key is configured.

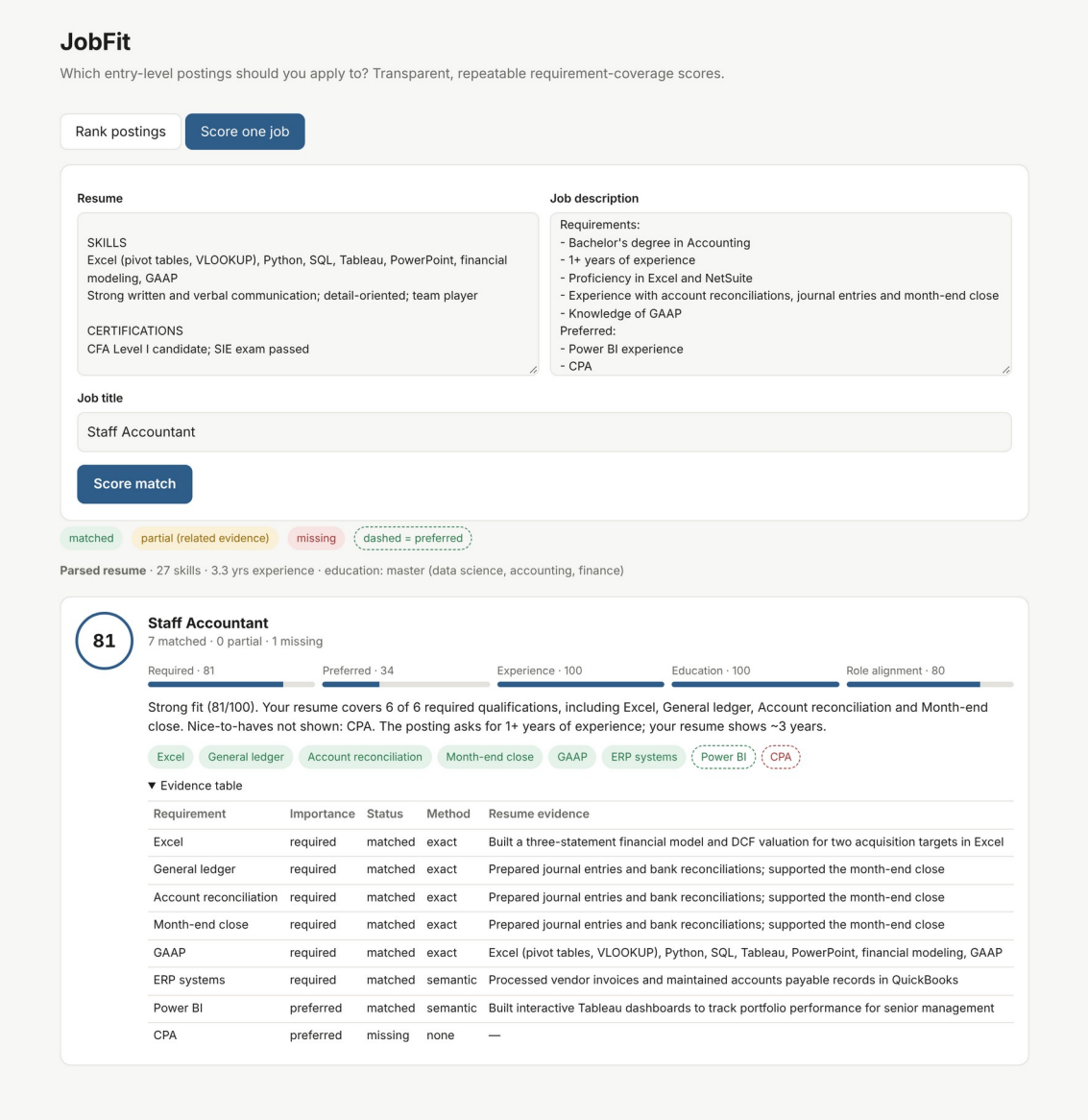


Figure 3. Web demo, single-posting mode: score, sub-score bars, explanation, colour-coded requirements and the evidence table. "Power BI" is supported semantically by the Tableau bullet; CPA is a certification and therefore exact-only.

3.6 API and deployment

The pipeline is served by FastAPI (extending the Module 3 service). Interactive documentation is generated at `/docs`.

Endpoint	Purpose
GET /health	Service status, index manifest, active extractor / LLM / embedding model
POST /rank	Rank all postings for a resume; filters: level, function, remote, location; top_k; number of explanations
POST /rank/upload	Same, with a PDF / DOCX / TXT upload (5 MB limit; 415 for unsupported types)
POST /match	One resume vs one pasted JD; returns match, JD profile and resume profile

Endpoint	Purpose
POST /jobs/{id}/match	Score a resume against a stored posting
GET /jobs/{id}	Posting metadata, extracted profile and description
POST /extract/jd, /extract/resume	Structured extraction only

Table 4. API endpoints.

The Docker image (Python 3.12-slim, ~316 MB compressed) installs pinned dependencies, downloads the embedding model at build time and copies the pre-built index, so the container starts and runs offline as a non-root user with a health check. `docker compose up --build` serves the API and demo on port 8000; setting `GEMINI_API_KEY` enables the LLM extractor/explainer, with the LLM cache on a volume. The test suite (26 tests) covers parsing, extraction, the formula, batch/pair agreement, determinism and every endpoint.

4. Evaluation

4.1 Set-up and weak labels

There is no public ground truth of resume–posting fit, so we use **weak labels**: a posting is relevant to a resume if its LinkedIn function tags match the resume's category (ACCOUNTANT → ACCT = 2, FIN = 1; FINANCE → FIN = 2, ACCT/ANLS = 1; BANKING → FIN = 2; INFORMATION-TECHNOLOGY → IT = 2; CONSULTANT → CNSL = 2, ANLS/IT = 1). For each of the 586 resumes every one of the 8,000 postings is scored, and we report nDCG@10 (graded), P@10 (any relevance), strict P@10 (grade 2) and MRR of the first grade-2 posting, with 95% confidence intervals over resumes. 48% of resume–posting pairs are relevant (29% strictly).

> **What the weak labels measure.** They reward *topical/category alignment*. They do not know whether a candidate meets a specific posting's requirements, so a method that only measures topic similarity is not disadvantaged, and a method that measures coverage is not rewarded for being right about requirements. We therefore complement them with targeted tests of the matcher (§4.4–4.6).

4.2 Ranking quality

Method	nDCG@10	P@10	Strict P@10	MRR
Random	0.343 ± 0.014	0.467	0.282	0.467
TF-IDF (keyword)	0.695 ± 0.025	0.832	0.632	0.707
Embedding doc-cosine	0.742 ± 0.022	0.854	0.687	0.792
Skill Jaccard (exact)	0.673 ± 0.025	0.791	0.612	0.706
Coverage only (required+preferred)	0.635 ± 0.024	0.766	0.561	0.705
JobFit (full)	0.701 ± 0.023	0.820	0.642	0.730
Hybrid: JobFit + doc-cosine (z-avg)	0.739 ± 0.022	0.852	0.684	0.773

Table 5. Ranking over all 8,000 postings for 586 resumes (weak labels).

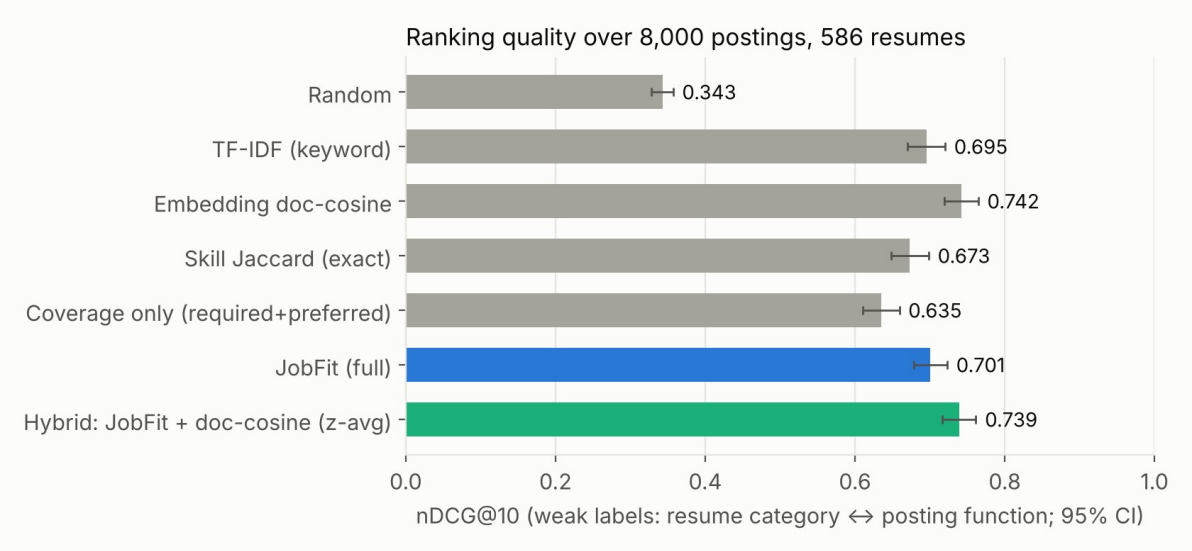


Figure 4. nDCG@10 with 95% confidence intervals.

JobFit (0.701) improves clearly on its closest structured baseline, exact skill overlap (0.673), and on the proposal's coverage-only formula (0.635); it is on par with TF-IDF (0.695, within the confidence interval). **It is below whole-document embedding similarity** (0.742). We report this plainly: a dense document embedding is an excellent *topic* retriever and the weak labels are topic labels. Combining the two (average of per-resume z-scores) recovers 0.739, indistinguishable from doc-cosine, while keeping JobFit's per-requirement evidence. The doc-cosine score, however, has no notion of requirements: it cannot say which qualifications are missing, does not change when a required skill is removed from a resume that stays on-topic, and its absolute value is not interpretable. In a deployed system we would use document similarity as a first-stage retrieval prior and JobFit's coverage score for ordering and explanation (§6).

4.3 Ablations

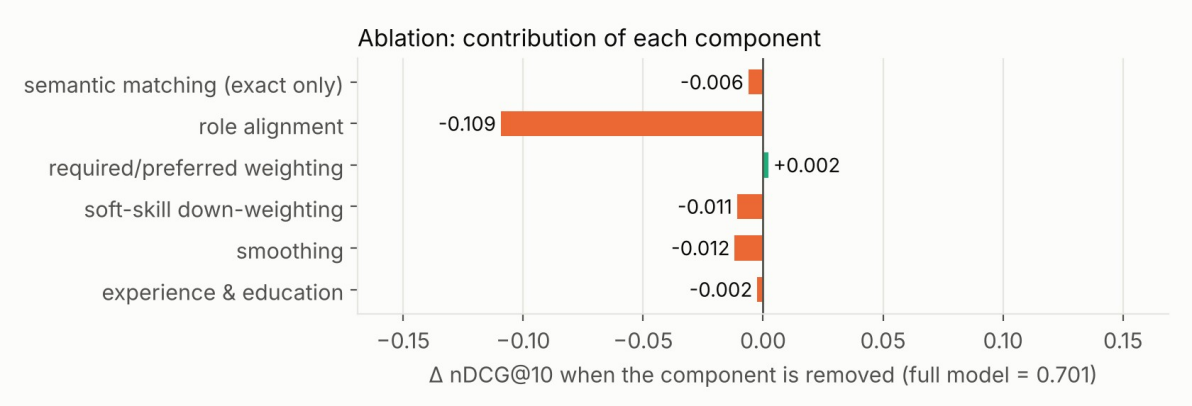


Figure 5. Change in nDCG@10 when one component is removed (negative = the component helps).

Role alignment is by far the most important component on these labels (-0.109 when removed), which is consistent with the labels being about job family. **Smoothing** (-0.012) and **soft-skill down-weighting** (-0.011) each help by about one point: without smoothing, postings with very few extracted requirements are over-ranked. **Semantic matching** adds a small amount (-0.006); its main

value is in paraphrase recall (§4.4) rather than category ranking. **Experience and education** (-0.002) and **required/preferred weighting** (+0.002) have no measurable effect here — expected, since these resumes are mostly experienced (experience is saturated) and weak labels say nothing about whether a specific requirement is mandatory. We kept them because they matter for the real use case (students with 0–2 years of experience) and for explanations, but we flag them as unvalidated by this evaluation.

4.4 Paraphrase benchmark for requirement matching

We wrote 42 test cases, each a requirement, a resume bullet that evidences it **without necessarily using the same words** (e.g. *Account reconciliation* ↔ "Matched bank statements to internal records every month and resolved discrepancies"), and a hard negative with similar wording ("Greeted visitors and managed the reception area"). A requirement counts as matched if its strength reaches the *partial* threshold.

Matcher	Recall (positives)	False-positive rate (negatives)	Pairwise accuracy
Keyword / taxonomy only	38%	0%	—
MiniLM, bare skill name	67%	0%	95%
MiniLM + descriptor (JobFit)	69%	0%	100%
bge-small + descriptor	86%	0%	100%

Table 6. Requirement matching on paraphrased evidence (keyword ∪ semantic). Pairwise accuracy: positive scores higher than its negative.

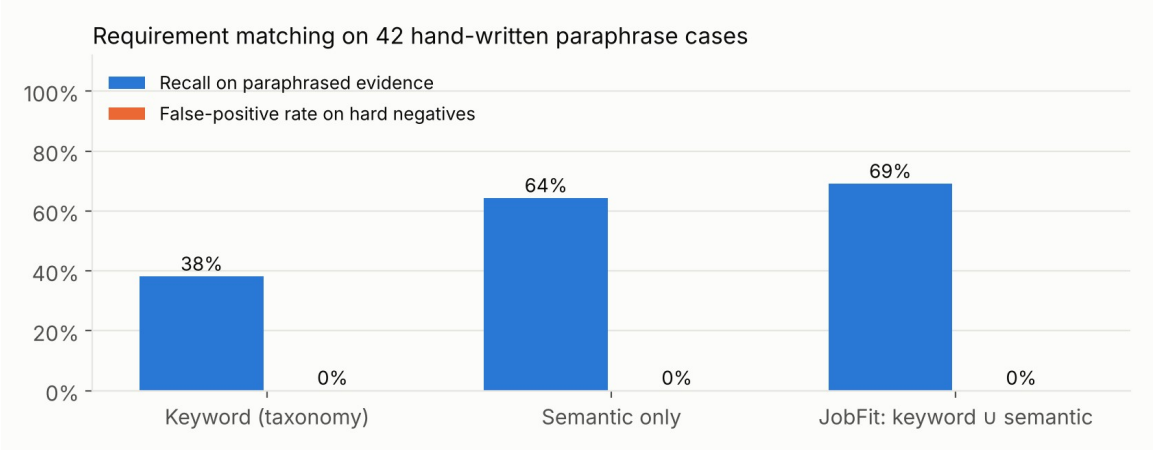


Figure 6. Keyword matching vs semantic matching vs JobFit's combination (MiniLM + descriptor).

Keywords find 38% of the evidence; JobFit finds 69% with no false positives, and always ranks the true evidence above the hard negative. Descriptors are responsible for a large part of the gain. bge-small finds more paraphrases (86%) on this small benchmark but separates the larger calibration set less well (AUC 0.900 vs 0.947); with 42 cases the difference is suggestive, and switching models is a one-line configuration change (`JOBFIT_EMBED_MODEL` plus re-calibration).

4.5 Counterfactual behaviour tests

A transparent score should move in the right direction when the resume changes. For 150 random resumes we took each one's top posting and (a) deleted every resume line that evidences one exactly-matched required skill, and (b) appended one line naming a required skill that was missing.

Edit	Cases	Mean score change	Score moved as expected
Remove evidence for a matched required skill	98	-6.7	100%
Add evidence for a missing required skill	17	+3.3	100%

Table 7. Counterfactual edits (fewer 'add' cases because top postings rarely had a missing required skill).

4.6 Repeatability and latency

Ranking the same resume three times produced **identical rankings and scores for 100% of 30 resumes** — by construction, since extraction is deterministic (rule-based, or LLM at temperature 0 with caching) and the formula has no randomness. A request that scores all 8,000 postings takes 156 ms on average (p50 146 ms, p95 246 ms) on a 4-core CPU, including parsing and embedding the resume.

4.7 Comparison with the plain-LLM approach

`eval/llm_baseline.py` measures the "paste resume + JD into a chatbot" approach on the same pairs: for each of 30 resume–posting pairs (a mix of top, mid and random postings) it asks the LLM for a 0–100 fit score five times at default sampling temperature, and reports the run-to-run standard deviation and range, the share of pairs whose score changed, latency per pair, the extrapolated time to rank all 8,000 postings, and the Spearman correlation with JobFit's score.

[To complete before submission] Our Gemini API key was rejected during the final runs, so the numbers for this subsection (and the LLM-extractor variant of the index) are not yet included. Run `GEMINI_API_KEY=… python eval/llm_baseline.py` and `python scripts/build_index.py --extractor llm`, then insert the values from `eval/results/llm_baseline.json` here. Independently of those numbers, two properties hold by construction: JobFit's score has zero run-to-run variance, and scoring all 8,000 postings costs one request (~150 ms), whereas the plain-LLM approach needs 8,000 LLM calls per resume.

5. Value added over off-the-shelf LLMs

	Plain LLM chat	JobFit
Scale	One posting per conversation	All 8,000 postings per request (~0.15 s), with filters
Repeatability	Score varies between runs	Identical output for identical input (tested)
Transparency	Opaque number + prose	Per-requirement status, method and quoted resume evidence; explicit formula
Faithfulness of explanation	Can mention anything	Generated only from the structured match output
Cost / availability	One LLM call per pair	JD extraction cached once; ranking needs no LLM; fully offline mode
Evaluation	None	Baselines, ablations, paraphrase, counterfactual

	Plain LLM chat	JobFit
		and stability tests

Table 8. What JobFit adds.

6. Limitations and ethical considerations

- **Weak labels.** Category labels measure topic, not fit; resumes in the Kaggle dataset are mostly experienced, not students. A small human-annotated set of resume–posting pairs (e.g. 3 annotators × 200 pairs, graded 0–3) would be the most valuable next evaluation and would let us fit the weights instead of setting them by hand.
- **Extraction coverage.** The rule-based extractor only knows its 115-skill taxonomy and section heuristics; requirements outside it (e.g. “experience with German credit regulations”) are invisible. The LLM extractor addresses this, at the cost of an API key, and its quality should be audited on a labelled sample of JDs.
- **Ranking vs document similarity.** On topic labels, document embeddings rank slightly better. A two-stage design — document-similarity retrieval of a few hundred candidates, then JobFit scoring — is a straightforward improvement.
- **Experience parsing** relies on date ranges and may miscount gaps, part-time roles or non-standard formats; over-qualification is not penalised.
- **Fairness.** The score uses only skills, experience and education; names, addresses and demographic proxies are ignored by the formula, but evidence text and embeddings can still carry bias (e.g. wording style). The score should help candidates prioritise, not be used by employers to screen people.
- **Privacy.** Uploaded resumes are processed in memory and not stored; with the LLM enabled, resume text is sent to the LLM provider, which users should be told about.

7. Conclusion and future work

JobFit turns a one-off chatbot interaction into a repeatable, explainable ranking service: structured extraction once per posting, calibrated semantic evidence matching, and an explicit scoring formula that ranks thousands of postings in a fraction of a second and shows exactly why each posting scored as it did. Our evaluation shows it clearly beats exact keyword-overlap scoring and recovers most paraphrased evidence without false positives, while being honest that topic-level document similarity remains a strong ranking signal on weak labels.

Next steps: (1) a human-labelled fit set to validate and fit the weights; (2) two-stage retrieval with document similarity; (3) LLM-extracted index and the measured plain-LLM comparison; (4) resume-improvement suggestions generated from the missing requirements of the top postings; (5) refreshing the job database from live postings.

Appendix A. Reproducing the results

Step	Command
Run the API	docker compose up --build (then open http://localhost:8000)

Step	Command
Download data	bash scripts/download_data.sh
Prepare data	python scripts/prepare_data.py
Build index	python scripts/build_index.py [--extractor llm]
Calibrate	python scripts/calibrate.py
Evaluate	python eval/evaluate.py && python eval/make_figures.py
Plain-LLM baseline	GEMINI_API_KEY=… python eval/llm_baseline.py
Tests	pytest -q

Table 9. Commands (see README.md).

Data sources. Kaggle Resume Dataset (snehaanbhawal/resume-dataset); Kaggle LinkedIn Job Postings 2023–2024 (arshkon/linkedin-job-postings). **Models.** sentence-transformers/all-MiniLM-L6-v2 (ONNX, via fastembed); optional Gemini 2.5 Flash-Lite through the OpenAI-compatible endpoint.